

GHOSTFACE — Cryptographic Inventory

Prepared by: Ghostface Limited (NZ) **Product:** GHOSTFACE — end-to-end encrypted mobile messaging app (iOS / Android) **Date:** 19 August 2026 **Purpose:** Supporting document for export-control classification review. Factual inventory of all cryptography used in the product, its sources, and its purpose. Prepared for review by legal counsel — this document states technical facts, not legal conclusions.

1. Summary

GHOSTFACE is a consumer end-to-end encrypted (E2EE) messaging and calling application, freely available to the general public through the Apple App Store and Google Play. Its encryption exists solely to protect users’ personal communications in transit and at rest. It performs no encryption function beyond that data-security purpose.

Every cryptographic algorithm used is a **standard, published algorithm** drawn from NIST FIPS standards, IETF RFCs, and peer-reviewed public specifications. The cryptography is implemented entirely using the widely-used, open-source @noble/* JavaScript libraries and the operating system’s own cryptographic random-number facilities. **No proprietary or secret cryptographic algorithm has been designed or implemented.**

The protocol design follows the publicly-documented Signal protocol family (X3DH + Double Ratchet), extended for post-quantum security using the same hybrid approach published by Signal (PQXDH) and Apple (iMessage PQ3).

2. Cryptographic primitives

Function	Algorithm	Standard	Source library (version)
Authenticated encryption (messages, media, storage)	ChaCha20-Poly1305	RFC 8439	@noble/ciphers 2.1.1
Key agreement (classical)	X25519 (ECDH on Curve25519)	RFC 7748	@noble/curves 2.0.1
Digital signatures	Ed25519	RFC 8032	@noble/curves 2.0.1
Post-quantum key encapsulation	ML-KEM-768	NIST FIPS 203	@noble/post-quantum 0.6.1
Hashing	SHA-256, SHA-512	FIPS 180-4	@noble/hashes 2.0.1
Key derivation (session)	HKDF-HMAC-SHA256	RFC 5869	@noble/hashes 2.0.1
Key derivation (PIN/passphrase)	PBKDF2-HMAC-SHA256, 600,000 iterations	RFC 2898 / NIST SP 800-132	@noble/hashes 2.0.1
Message authentication	HMAC-SHA256	RFC 2104 / FIPS 198-1	@noble/hashes 2.0.1

Function	Algorithm	Standard	Source library (version)
Random number generation	OS CSPRNG — SecRandomCopyBytes (iOS), java.security.SecureRandom (Android)	Platform-provided	react-native-get-random-values 1.11.0, expo-crypto
Key storage	iOS Keychain / Android Keystore	Platform-provided	expo-secure-store

All of the above are standard published algorithms. None is modified.

3. Protocols and how the primitives are combined

GHOSTFACE composes the standard primitives above into the publicly-documented Signal protocol design:

Initial key agreement — X3DH, extended to PQXDH. New sessions are established using Signal’s Extended Triple Diffie-Hellman (X3DH) over X25519, with prekey signatures verified via Ed25519. Where both parties support it, the handshake additionally performs an ML-KEM-768 key encapsulation and mixes the resulting shared secret into the session key — identical in structure to Signal’s published PQXDH specification. The app requires this post-quantum step for all new sessions.

Ongoing messaging — Double Ratchet. Established sessions use Signal’s Double Ratchet algorithm: a Diffie-Hellman ratchet (X25519) plus symmetric-key ratchets, with per-message keys deriving forward secrecy and post-compromise security. Each message is encrypted with ChaCha20-Poly1305 under a per-message key.

Key-derivation functions. Root- and chain-key derivation uses HKDF-HMAC-SHA256 (RFC 5869) throughout — the standard construction used by the Signal protocol itself.

Post-quantum continuous rekey (kdfRkPQ). For PQ-enabled sessions, the root-key update mixes the post-quantum shared secret into the ratchet. This is implemented as a single HKDF-HMAC-SHA256 call whose input keying material is the concatenation of (a) the X25519 Diffie-Hellman output and (b) the ML-KEM-768 shared secret, with a distinct domain-separation label. **It is standard HKDF-SHA256 applied to two standard shared secrets — it defines no new cryptographic algorithm or primitive.** Its purpose and structure mirror the continuous post-quantum rekeying published in Apple’s iMessage PQ3 design.

Data at rest. Locally-stored data (message history, keys) is encrypted with ChaCha20-Poly1305 under a master key held in the OS keychain (iOS Keychain / Android Keystore). PIN/passphrase-derived keys use PBKDF2-HMAC-SHA256 at 600,000 iterations (OWASP-recommended).

4. Nature and availability

- **Function:** data confidentiality and integrity for personal communications only.

- **Availability:** free consumer app, distributed to the general public through mainstream app stores; no restriction on class of user.
- **Cryptographic novelty:** none at the primitive level. The only non-verbatim element (`kdfRkPQ`) is a standard-HKDF composition of two standard shared secrets, matching published hybrid-PQ designs.
- **Key length / strength:** as specified by the underlying standards (e.g. 256-bit symmetric, Curve25519, ML-KEM-768 / NIST security category 3).
- **Open-source dependencies:** all cryptography via the public `@noble/*` libraries; the app implements no cryptography in a closed or secret manner.

This inventory reflects the state of the codebase as of 19 August 2026 and is provided to assist counsel's export-control classification assessment. It is a technical description prepared by the company, not a legal classification.